

Compliance & Regulatory Q&A Assistant

A grounded, citation-first RAG system — codebase explained

What this document covers: the end-to-end architecture, the exact job of every source file, the key functions inside each, and how a question flows from text to a cited answer (or a refusal).

Stack: FAISS (dense) + BM25 (lexical) hybrid retrieval · cross-encoder reranker · local Flan-T5 generation · FastAPI + CLI · single-file web dashboard. Fully local, no API keys.

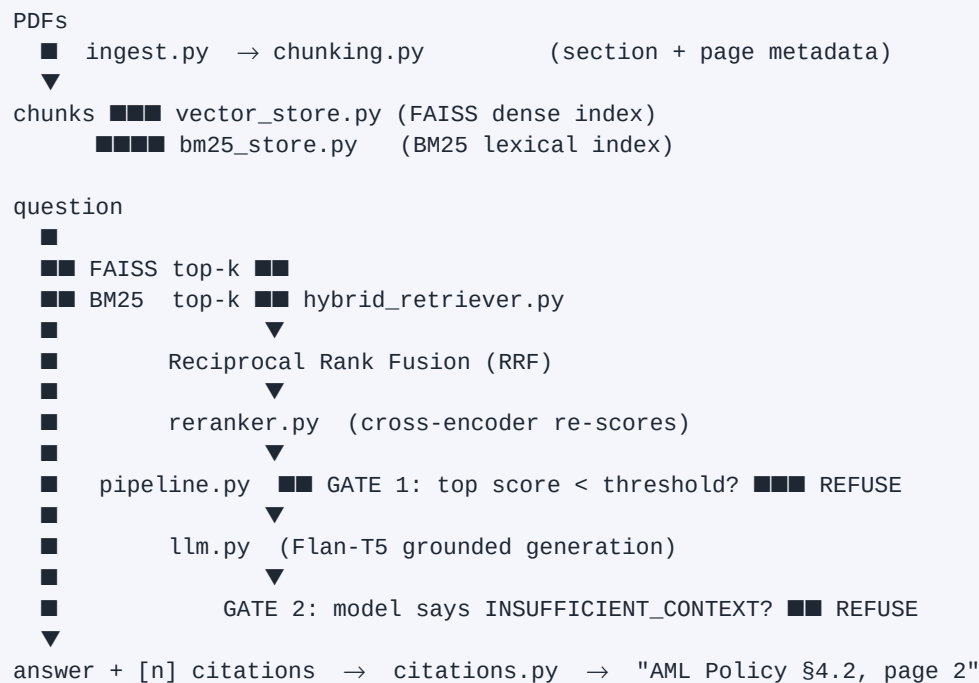
1 · The big picture

The system answers questions about a corpus of compliance/regulatory PDFs and, crucially, **cites the exact section and page** behind every answer. When the corpus does not contain the answer, it **refuses instead of guessing**. That anti-hallucination behaviour is the core value: regulated users need "show me the source," not a confident fabrication.

Two ideas do most of the work:

- **Hybrid retrieval.** Dense embeddings (FAISS) catch meaning and paraphrase; BM25 catches exact terminology — statute numbers, defined terms, acronyms. Their results are fused, then a cross-encoder re-scores the finalists.
- **Two-gate refusal.** Gate 1 checks the reranker's confidence; Gate 2 lets the generator itself declare the context insufficient. Both must pass to return an answer.

Pipeline flow



2 · Project layout

Every file and what it is responsible for.

File	Responsibility
src/rag/config.py	All tunable settings (models, chunk sizes, top-k, refusal threshold, paths). Reads .env.
src/rag/schemas.py	Plain data classes: Chunk, ScoredChunk, Citation, Answer. The shared vocabulary.
src/rag/chunking.py	Splits page text into section-aware chunks, tagging each with §section and page.
src/rag/ingest.py	Reads PDFs → per-page text → chunks with document metadata + stable IDs.
src/rag/embeddings.py	Local sentence-transformers embeddings (lazy-loaded, L2-normalized).
src/rag/vector_store.py	FAISS dense index + parallel chunk metadata; build / save / load / search.
src/rag/bm25_store.py	BM25 lexical index over the same chunks; build / save / load / search.
src/rag/reranker.py	Cross-encoder that re-scores (query, passage) pairs. Powers the refusal score.
src/rag/hybrid_retriever.py	Runs dense + BM25, fuses with RRF, then reranks. The retrieval brain.
src/rag/llm.py	Flan-T5 grounded generation with a strict, cite-or-refuse prompt.
src/rag/citations.py	Builds [n] citation markers and keeps only the ones the answer used.
src/rag/pipeline.py	Orchestrates everything and enforces the two refusal gates.
src/rag/api.py	FastAPI app: serves the dashboard at / and the /query + /health endpoints.
src/rag/cli.py	Command-line `build` and `ask` commands.
web/index.html	Single-file browser dashboard (served by the API at /).
scripts/generate_sample_pdfs.py	Creates the illustrative compliance PDF corpus.
scripts/build_index.py	Convenience wrapper to ingest + build both indexes.
eval/evaluate.py	Retrieval Hit@k / MRR and refusal-accuracy metrics.
tests/	pytest suite for chunking, fusion, citations, and both refusal gates.

3 · Module-by-module walkthrough

config.py — one place for every knob

A dataclass of settings, each overridable by an environment variable or a .env file, so you never hunt through code to change a model or a threshold.

- **Models:** embedding_model, reranker_model, generator_model (Hugging Face IDs).
- **Chunking:** chunk_size (900 chars), chunk_overlap (150).
- **Retrieval:** dense_top_k, bm25_top_k, fusion_top_k, rerank_top_k, rrf_k.
- **refusal_threshold (0.15):** the single most important tuning dial — raise it for stricter grounding (more refusals), lower it for higher recall.

schemas.py — the shared vocabulary

Small dataclasses passed between stages so every module agrees on shape. The important one is Chunk, which carries provenance and can format its own citation.

- **Chunk:** text + doc_id, doc_title, section, page — plus .citation() → "AML Policy §4.2, page 2".
- **ScoredChunk:** a Chunk with a relevance score and per-stage debug info.
- **Answer:** the final object — question, answer, refused flag, citations, top_score.

Chunk.citation() — how a source tag is produced:

```
def citation(self) -> str:
    sec = self.section if self.section.startswith("$") else f"${self.section}"
    return f"{self.doc_title} {sec}, page {self.page}"
```

chunking.py — section-aware splitting

Compliance documents are hierarchical (§1, §1.1, §4.2 ...). A regex detects numbered headings and starts a new segment at each, so every chunk inherits the correct section label and the page it began on. Long sections are then split into overlapping windows that break on sentence boundaries.

- A heading like "4.2 EDD Measures" becomes the label §4.2 and anchors all following text until the next heading.
- _split_with_overlap keeps ~150 chars of overlap so a fact split across a boundary is never lost.
- This is why citations resolve precisely: the section+page travel with the text into the index.

```
SECTION_RE = re.compile(r"^\s*(?:$s*)(\d+(?:\.\d+)*)\s+(.+)s*$")
# a matched heading flushes the current buffer and starts a new
# (section, page) segment; body lines accumulate under it.
```

ingest.py — PDFs to chunks

Turns a folder of PDFs into a flat list of Chunk objects. Each PDF is read page-by-page (pypdf), handed to the chunker, and every chunk gets a stable SHA-1 id plus a human title derived from the filename (AML-Policy.pdf → "AML Policy").

- `read_pdf_pages()` → per-page text with 1-based page numbers.
- `ingest_dir()` → all chunks across every PDF in `data/raw`.

embeddings.py — dense vectors, locally

Wraps a sentence-transformers model (all-MiniLM-L6-v2 by default). Loaded lazily on first use and cached. Vectors are L2-normalized so an inner product equals cosine similarity — which is exactly what the FAISS index expects.

- `embed_texts()` for the corpus, `embed_query()` for a single question.
- No API key, no network after the first model download.

vector_store.py — the FAISS dense index

Builds a FAISS IndexFlatIP over the normalized chunk vectors (inner product = cosine). Because FAISS returns integer ids, the Chunk objects are pickled alongside the index so those ids map straight back to source passages for citation. Persisted to storage/ so you build once and query many times.

- `build()` embeds all chunks and adds them to the index.
- `save()/load()` persist and restore both the index and the metadata.
- `search()` returns (Chunk, score) pairs for the nearest passages.
- This adapter is deliberately thin — swap FAISS for pgvector by reimplementing these four methods.

bm25_store.py — the lexical index

A classic BM25 index (rank-bm25) over the same chunks. It complements the embeddings by matching exact tokens — the '§4.2's and 'PEP's that dense vectors blur together. Same build/save/load/search shape as the FAISS store.

- A custom tokenizer keeps §, digits and dots so section references survive tokenization.

reranker.py — the sharp second opinion

A cross-encoder reads the query and a passage *together* and outputs a single, well-calibrated relevance score. That calibration is what makes the refusal decision trustworthy: if even the best passage scores low, the answer genuinely isn't in the corpus.

- `rerank()` scores every candidate, sorts, and returns the top-k as ScoredChunk objects.
- Scores are squashed through a sigmoid into 0–1 so the refusal threshold is intuitive.

hybrid_retriever.py — dense + lexical + rerank

The retrieval brain. It queries both stores, fuses their rankings with Reciprocal Rank Fusion, then reranks the fused pool. RRF is used instead of blending raw scores because cosine and BM25 scores live on different scales — RRF needs only each item's rank, which makes fusion robust and near parameter-free.

- Each retriever contributes $1 / (k + \text{rank})$ to a chunk's fused score; agreement between retrievers floats a passage to the top.

- Output: the reranked top-k ScoredChunks handed to the pipeline.

```
scores[cid] += 1.0 / (k + rank + 1)  # from dense ranking
scores[cid] += 1.0 / (k + rank + 1)  # from BM25 ranking
# then sort by fused score and rerank the survivors
```

llm.py — grounded generation

Local Flan-T5 generation behind a deliberately strict prompt: answer ONLY from the numbered context passages, cite each claim with its [n], and if the passages don't contain the answer reply with the exact sentinel INSUFFICIENT_CONTEXT. That sentinel is the model-side half of the refusal mechanism.

- `build_context()` numbers the passages and prefixes each with its citation.
- `generate()` returns the model's answer text (or the sentinel).

The core of the grounding prompt:

```
Answer the QUESTION using ONLY the numbered CONTEXT passages.
Every claim must be cited with its [number]. If the passages do
not contain the answer, reply with exactly: INSUFFICIENT_CONTEXT
```

citations.py — provenance you can trust

Attaches [1], [2], ... markers to the retrieved passages and, after generation, keeps only the sources the answer actually referenced. If the model cited nothing, it falls back to showing all supporting passages — still grounded, never bare.

- `build_citations()` → Citation objects with source tag, section, page and a short snippet.
- `filter_to_used()` → prunes to the markers that appear in the answer text.

pipeline.py — orchestration & the two gates

Ties the whole flow together and enforces refusal. This is where the product promise lives.

- **Gate 1 (retrieval):** if there are no hits, or the best reranker score < `refusal_threshold`, refuse *without even calling the generator*.
- **Gate 2 (generation):** if the model emits `INSUFFICIENT_CONTEXT` (or nothing), refuse.
- Only when both gates pass does it return an answer with filtered citations and the top score.

```
if not scored or top_score < settings.refusal_threshold:
    return Answer(..., refused=True)          # GATE 1
raw = generate(question, scored)
if INSUFFICIENT in raw or not raw.strip():
    return Answer(..., refused=True)          # GATE 2
return Answer(..., refused=False, citations=filter_to_used(raw, ...))
```

api.py & cli.py — the two front doors

Same pipeline, two interfaces. The API loads the pipeline once at startup and reuses it; the CLI is handy for scripting and quick checks.

- **api.py:** GET / serves the dashboard, POST /query returns the Answer as JSON, GET /health for liveness. A 503 with a clear message is returned if the index hasn't been built yet.

- **cli.py:** ``build` ingests + indexes; `ask "...` prints the answer and sources in the terminal.`

web/index.html — the dashboard

A dependency-free single page served by the API. It shows a search box with example questions, a grounded/refused badge, the retrieval confidence score, and each cited source with its §section and page. A status dot confirms the API is reachable.

4 · Life of a question (worked example)

Tracing "what triggers enhanced due diligence?" end to end:

1. Retrieve (dense)	The question is embedded; FAISS returns the nearest chunks — the §4 and §4.1 passages of the AML Policy rank highly on meaning.
2. Retrieve (lexical)	BM25 independently scores on the words 'enhanced', 'due', 'diligence' and also surfaces §4 / §4.1.
3. Fuse (RRF)	Both lists agree on §4.1, so RRF floats it to the top of the fused candidate pool.
4. Rerank	The cross-encoder reads (question, passage) pairs and gives §4.1 a high calibrated score (~0.9).
5. Gate 1	$0.9 \geq 0.15$ threshold → passes. Proceed to generation.
6. Generate	Flan-T5 answers from the numbered passages and cites [1]; INSUFFICIENT_CONTEXT is not emitted.
7. Gate 2	No sentinel → passes.
8. Cite	citations.py keeps [1] and renders 'AML Policy §4.1, page 2' with a snippet.

Now the counter-case: "What is the corporate tax rate in Ireland?" No passage scores above 0.15, so **Gate 1 refuses immediately** — the generator is never called, and the user is told the corpus doesn't cover it.

5 · Running it

```
# 1. install (first run also downloads ~500MB of models)
python -m venv .venv && .venv\Scripts\activate      # Windows
pip install -r requirements.txt

# 2. build the FAISS + BM25 indexes from data/raw
python -m src.rag.cli build

# 3a. ask from the terminal
python -m src.rag.cli ask "What triggers enhanced due diligence?"

# 3b. or launch the API + dashboard
uvicorn src.rag.api:app --reload      # then open http://127.0.0.1:8000/

# checks
pytest -q          # 12 unit tests, no model download needed
python -m eval.evaluate  # retrieval Hit@k + refusal accuracy
```

Where to change behaviour: retrieval quality lives in `hybrid_retriever.py` and `reranker.py`; how strict the refusal is lives in one number — `refusal_threshold` in `config.py`; answer style lives in the prompt in `llm.py`.